ACoT-VLA：用动作语言进行推理

历史论文按新提示词重制版 · CVPR 2026 · 2026-09-23

对应离线阅读包：解压ZIP后双击index.html。本PDF完整收录13项学习简报，并重点展开创新点、三项核心算法、数学公式和固定commit源码。

1. 基本信息

项目	内容
标题	ACoT-VLA: Action Chain-of-Thought for Vision-Language-Action Models
作者	Linqing Zhong, Yi Liu, Yifei Wei, Ziyu Xiong, Maoqing Yao, Si Liu, Guanghui Ren
机构	AgiBot；Beihang University
会议/版本	CVPR 2026；arXiv:2601.11404v2，2026-03-30，15页
资源	arXiv · CVF · 固定源码
代码状态	当前仓库已公开核心 acot_vla.py、训练配置、训练/推理入口；README仍把部分checkpoint列为TODO

2. 分类标签

VLA、机器人策略学习、动作空间思维链、flow matching、视觉语言模型、动作chunk、显式/隐式推理、sim-to-real。

3. 个性化推荐理由

这篇把你已有的 Qwen2.5-VL LoRA/SFT 与未来 VLA Agent
连接到真正可执行动作：不是让模型输出一段“解释文字”，而是先产生粗动作轨迹，再让精细动作头以其为条件去噪。你做 GRPO 时关注中间推理是否真的改善最终目标；ACoT
给出一个很好的对照：中间变量与最终输出同处动作空间，因此监督密度和可执行性更强。对 VGGT/Depth Pro/YOLO
项目，它还提示可把3D目标中心、朝向和接触点编码进粗轨迹，而不是只拼到视觉token里。

4. 一句话问题定义

面对“把杯子拿起并倒水”这类任务，VLM知道杯子是什么，却不天然知道末端执行器接下来10步怎样移动；ACoT-VLA先生成一条粗动作意图链，再用它约束精细动作生成，从而缩小语义理解与运动控制之间的鸿沟。

5. 方法流程

- 输入最多三路224×224 RGB、语言token和可选机器人状态；SigLIP+Gemma形成前缀token与逐层KV cache。
- EAR从高斯噪声出发，以flow matching迭代生成粗动作 [B,H_ref,A]。
- IAR逐层读取KV [B,L,T,D]，用一条可学习query、降维多头注意力压成 [B,L,D_a]。
- 最终动作头对显式粗轨迹和隐式KV摘要分别cross-attention，拼接后self-attention融合。
- 动作头从另一份高斯噪声迭代去噪，输出 [B,H,A]；训练时EAR与动作头各有一个velocity MSE。

6. 真正创新点：已有方法 vs ACoT-VLA

维度	旧方法/瓶颈	本文修改	为什么有效	论文证据
中间推理空间	语言子任务或目标图像；与低层动作异质	把粗动作序列本身作为CoT	同域条件减少语义到运动的歧义	LIBERO Long 92.4→96.6/97.0；EAR消融+1.4个百分点
显式指导	动作头一次性从观察去噪	EAR先生成参考轨迹，动作头再条件生成	类似self-conditioning，提供运动学路径先验	LIBERO-Plus EAR 75.7→83.7
隐式指导	只使用VLM最终表示	从每层KV提取动作相关潜变量	利用跨层可供性/语义，同时降维去噪	IAR 75.7→80.4；Downsample优于Query
融合	简单拼接或单一条件	双cross-attention+融合attention	分别对齐显式/隐式先验，再联合	EAR+IAR 达84.1/98.5
监督	单个精细动作loss	粗轨迹loss+精细动作loss，teacher forcing	粗任务提供额外监督且不让早期错误污染动作头	规模/去噪匹配消融仍显示EAR收益

不是本文创新：SigLIP、Gemma/PaliGemma、π0.5动作专家、flow matching、cross-attention、teacher forcing本身均为已有技术；本文贡献是把它们组织为动作空间推理链，以及EAR/IAR/AGP的数据流。

7. 核心算法一：EAR 粗动作 flow matching

1）问题与衔接

EAR先从观察和指令合成粗动作轨迹，供最终动作头参考。它位于VLM前缀编码之后、AGP之前。

2）输入输出与形状

B批量；H_ref=15默认参考时域；A动作维（配置默认32，LIBERO实际由数据管线确定）；噪声与目标均为[B,H_ref,A]。VLM前缀KV在代码整理为 [B,L,T,D]。EAR动作token为 [B,H_ref,D_r]，输出velocity同形后投影到A维。

3）预处理/采样/前向

动作与观测先按数据norm stats归一化。训练采样 t~Beta(1.5,1)*0.999+0.001，噪声 ε~N(0,I)，插值 x_t=tε+(1-t)a，目标速度 u_t=ε-a。动作投影后加入sin-cos timestep；π0.5路径用AdaRMSNorm注入时间。mask允许动作token看全部前缀和同一动作段。

4）训练与梯度

EAR预测 v_ref，MSE为 mean((u_ref-v_ref)^2)。代码训练最终动作头时使用真实 coarse_actions 作为显式条件（teacher forcing），所以动作头loss不会经预测粗轨迹反传进EAR；EAR由自己的MSE训练。配置冻结Gemma主LLM，但不冻结视觉与两个动作专家。

5）推理

从高斯噪声开始，默认10步，dt=-1/num_steps，从t=1积分到0：x←x+dt·v(x,t)。生成的粗轨迹随后成为最终动作头条件。

6）小例子

标量动作a=0.2、噪声ε=1.0、t=0.25，则x_t=0.25×1+0.75×0.2=0.40，目标速度u=0.8。若网络预测0.7，该样本平方误差0.01；推理时dt=-0.1，则x从0.40更新到0.33。

```
# 自行编写的示意伪代码，不是作者代码
t = Beta(1.5,1); eps = randn_like(coarse)
x = t*eps + (1-t)*coarse
v = EAR(prefix_kv, x, t)
loss_ref = mean((eps-coarse-v)**2)
# inference
x = randn(B,H_ref,A)
for k in range(10): x = x - 0.1*EAR(prefix_kv,x,1-k/10)
```

7）源码映射

compute_loss L694-787；EAR采样 L795-860；EAR token/time L576-597。

8）论文-实现差异

论文把EAR写成轻量Transformer逐层cross-attend VLM KV；公开实现复用一个PaliGemma多专家LLM，把前缀KV作为cache并把粗动作专家作为第二分支。论文写λ1=λ2=0.5；代码直接把两个均值MSE相加，并注释“因平衡因子为0.5，以下loss等价”，这只在整体常数缩放不影响所关心优化方向时成立，数值上并非原公式的0.5倍。

8. 核心算法二：IAR 从逐层KV提取隐式动作先验

1）问题与衔接

IAR避免只用VLM最终层；它从每层KV中提取“可抓区域、动作动词、交互关系”等隐式线索，供最终动作token查询。

2）输入输出与shape

KV从LLM返回 [L,B,T,1,D]，去掉单KV头维并重排为 [B,L,T,D]。当前DownsampleExtractor每层一条query（Q=1），投影到 d'=D/2（注意论文写128，代码用 head_dim//2），分成h个头，每头d_h=d'/h。输出为 [B,L,D_a]。

3）前向顺序

每3层共享一组Q/K/V/out投影，但每层有自己的可学习query。Q为 [1,h,1,d_h] 并tile到B；K,V为 [B,h,T,d_h]；分数 [B,h,1,T] 在T维softmax；加权V得到 [B,h,1,d_h]，压掉query维、拼头并投影到动作专家宽度。

4）训练与梯度

IAR没有独立标签或loss，梯度从最终动作velocity MSE穿过AGP、IAR输出投影、attention和可学习query。Gemma是否更新由freeze_filter决定；公开LIBERO配置冻结主LLM，因此梯度只更新提取器、动作专家和未冻结模块。

5）推理

前缀只前向一次，KV被缓存；IAR也只抽取一次，供全部最终去噪步重复使用，避免每步重算视觉语言主干。

6）形状例子

设B=2、L=18、T=300、D=256，d'=128、h=8，则每头16维；QK⊠得到[2,8,1,300]，softmax后乘V得[2,8,1,16]，拼头为[2,128]；18层堆叠为[2,18,D_a]。

```
# 自行编写示意
for layer in range(L):
q = project(learnable_q[layer]) # [1,h,1,dh]
k,v = project(K[:,layer]), project(V[:,layer])
w = softmax(q @ k.transpose(-1,-2) / sqrt(dh), dim=-1)
z[layer] = out((w @ v).merge_heads())
return stack(z, dim=1) # [B,L,Da]
```

7）源码映射

DownsampleExtractor L156-239；模块构造 L439-483；训练KV重排 L749-756。

8）差异与误区

论文说W矩阵按层记作W^(i)，代码每3层共享一组投影；论文 d'=128，当前代码设置为PaliGemma head_dim的一半，需以实际Gemma配置核算，不能默认恒等于128。IAR提取的是“动作相关潜变量”，不是可读的语言CoT。

9. 核心算法三：AGP 双条件融合与最终去噪

1）问题与衔接

显式轨迹长度H_ref与最终动作长度H不同，隐式先验又按VLM层排列；AGP让每个最终动作token主动查询两类指导并对齐。

2）输入输出与shape

最终带噪动作token X[B,H,D_a]；显式轨迹投影 E[B,H_ref,D_r]；隐式先验 I[B,L,D_a]。两次cross-attention输出均为 [B,H,D_a]，拼接、线性投影、融合attention后仍为 [B,H,D_a]，最后线性层输出velocity [B,H,A]。

3）前向与mask

对显式和隐式条件分别做Q=X、K/V=E或I。实现先把X与每个attended结果拼接并投影成两条D_a特征，再拼成2D_a，送入Unified AttentionModule做自注意式融合。动作段的ar_mask只有第一token标True，其余False，与π0流匹配动作块的双向可见方式一致；前缀KV只读。

4）训练与梯度

最终目标同样为 u=ε-a 的velocity
MSE。梯度流过动作输出层、融合模块、两个cross-attention、IAR；显式分支训练时读取真实coarse action，因此不会把动作头梯度送入EAR预测过程。总损失还额外包含EAR自身MSE。

5）推理

先完整运行EAR
10步得到粗轨迹，再运行最终动作头10步；因此是串行的“两阶段×去噪步数”，不是一次前向。两阶段共用只计算一次的VLM prefix KV cache。

6）例子

若H=10、H_ref=15、L=18、D_a=1024，则第4个精细动作token会分别对15个粗动作token和18个层级先验产生注意力权重，而不是要求位置一一对应。这使“参考轨迹更长/shift不同”仍能提供条件。

```
# 自行编写示意
s_ex = cross_attn(x_action, coarse_tokens, coarse_tokens)
s_im = cross_attn(x_action, implicit_tokens, implicit_tokens)
h_ex = proj(concat(x_action, s_ex))
h_im = proj(concat(x_action, s_im))
h = fusion_attn(concat(h_ex, h_im))
velocity = action_out(head(prefix_kv, h, t))
```

7）源码映射

UnifiedAttentionModule L242-263；双分支融合 L620-647；最终动作采样 L863-896。

8）差异与注意

论文式(11)写直接Self-Attn([S_ex;S_im])；代码在两路S与原动作token分别拼接并线性投影后，再拼接融合，源码注释明确承认该维度变换未在论文显式写出。README的112ms是单次端到端报告值；不要把它误读为单个denoise step。

10. 数学公式逐式详解

式(1)-(2)：条件策略分解

式(1)定义H步策略输出；式(2)用乘法法则把“先产生指导g，再在g条件下产生动作”写成联合概率。它不是额外loss，而是建模视角。ACoT的关键是把g扩展到动作域。若两个可能粗轨迹概率各0.6/0.4，而精细动作在其条件下概率0.8/0.3，则边缘动作概率为0.6×0.8+0.4×0.3=0.60。

式(3)-(6)：EAR

KV的层维N对应18层；式(4)把时间self-attention和逐层VLM cross-attention相加，输出保持[B,H_ref,D_r]；式(5)残差FFN稳定深层训练；式(6)表示条件去噪分布。公开代码实际用共享prefix KV cache和Gemma动作专家实现。

式(7)-(8)：IAR注意力

降维将计算从O(MTd)降到O(MTd')，d'≪d。标准注意力为 softmax(QK⊠/√d_h)V；softmax沿T维，使每条query在全部视觉/语言token间归一。除以√d_h避免点积方差随维度增大导致softmax饱和。例：分数[1,2] softmax约[0.269,0.731]，若V=[0,1]，输出0.731。

式(9)-(11)：双cross-attention

Q_action的每个位置分别从粗轨迹和层级隐式先验检索。两路输出shape均[B,H,D_a]；拼接后是[B,H,2D_a]。使用注意力而不是固定位置相加，允许H、H_ref、L不同，并让每个精细时刻选择不同参考片段。

式(12)与实现中的flow matching

论文总loss是λ1L_ref+λ2L_head。实现的直线路径为 x_t=tε+(1-t)a，速度目标 u=ε-a，MSE使网络学到从数据t=0指向噪声t=1的向量场；推理以负dt从1积分回0。参数更新：EAR MSE更新粗专家；最终MSE更新动作头、AGP和IAR；teacher forcing阻断最终loss经“预测粗轨迹”污染EAR。

11. 实验、局限与适用范围

关键证据：LIBERO平均98.5%，Long 97.0%；LIBERO-Plus zero-shot 86.6%、SFT 88.0%；VLABench IS/PS 63.5/47.4；真实四任务综合66.7%。模块消融显示EAR和IAR单独有效，合并最佳；匹配总参数/去噪预算后EAR仍提升，排除“只是模型更大”。

作者局限：额外推理增加资源开销；action chunk缺少显式3D几何，限制更高层空间推理。源码/复现局限：仓库当前包含核心实现与配置，但README仍把官方checkpoint列为TODO；公开配置里有作者本地绝对数据/权重路径，需要替换。GitHub issue中已有用户报告LIBERO-Plus复现差距，故论文数字不能视为当前仓库开箱即得。

适用：动作维度和控制频率固定、可获得足量示范的操作任务。跨机器人时需重新定义动作空间、归一化和时域，粗动作并非天然跨本体。

12. 前置知识与55分钟学习路线

前置知识：（1）action chunk：一次预测连续H步；（2）flow matching：学习把噪声沿向量场运输到数据；（3）KV cache：VLM每层token的键和值；（4）cross-attention：一个序列以Q从另一个序列K/V检索；（5）teacher forcing：训练用真中间量替代模型预测。

- 0-8分钟：看图1，明确语言/视觉/动作CoT差异。
- 8-18分钟：看图2和式(1)-(6)，画出EAR输入输出。
- 18-30分钟：手推flow matching的x_t、u_t和Euler反向积分。
- 30-40分钟：对照DownsampleExtractor，走一遍[B,L,T,D]→[B,L,D_a]。
- 40-48分钟：读表4、9、10，判断收益是否来自参数量。
- 48-55分钟：看源码teacher forcing与两段while_loop，回答：训练-推理差距会怎样累积?

13. 代码任务、面试题与迁移实验

代码阅读优先级：① acot_vla.py L694-900：训练与两阶段采样；② acot_vla.py L156-263：IAR与统一注意力；③ config.py L1583-1674：LIBERO/VLABench/Plus配置。权重初始化映射见 weight_loaders.py L56-81。

最小运行路线：官方README要求uv环境、LeRobot格式数据、先compute_norm_stats，再train.sh，server.sh提供策略服务。论文完整训练为8×H100；单次推理报告RTX4090。由于官方checkpoint仍未发布，不把“克隆仓库+启动脚本”说成可复现论文结果。

面试题1：ACoT与语言CoT本质区别？答：中间变量与输出是否同域；动作CoT直接提供运动学轨迹，减少语义-控制映射歧义，但牺牲可读性和跨本体通用性。
面试题2：为什么teacher forcing既稳定又有风险？答：训练时动作头看到真实粗轨迹，梯度更稳；推理时看到EAR预测，存在exposure bias，EAR误差可能在第二阶段放大。

迁移到VGGT/Depth Pro/YOLO：在EAR粗动作token旁增加对象中心、3D框yaw、接触候选点和可行抓取轴，形成[B,H_ref,A+G]，G来自VGGT/Depth Pro几何与YOLO实例。A组原ACoT；B组仅把3D特征拼到VLM token；C组把3D几何直接加入粗动作CoT。预期C组在相机视角变化和未见纹理下更稳。验证假设：几何误差是否会比视觉语义误差更严重地误导EAR；指标用成功率、碰撞率、末端轨迹误差和跨视角性能。对Qwen2.5-VL/VLA Agent，可先SFT动作CoT，再只对最终任务成功做小规模GRPO，但应给EAR单独稳定奖励或KL约束，避免奖励黑客把粗轨迹变成不可执行编码。